

Engineering a Containerized SOC Lab with Splunk

There's a difference between running a SIEM and actually understanding how detection works.

Most SOC lab setups focus on dashboards - ingest logs, run a few queries, and stop there. That approach shows visibility, but it doesn't explain how real attack signals emerge from raw telemetry.

That wasn't enough for me.

I wanted to understand how authentication logs evolve into detection signals - how repeated failures, timing patterns, and source behavior translate into indicators of compromise.

So instead of just using Splunk, I engineered a **containerized SOC lab using Docker**, focused on building a **detection pipeline**, not just a dashboard.

Engineering Goals

To move beyond a basic SIEM setup, I defined three core objectives:

1. **Containerized Deployment**
Build a lightweight and reproducible SIEM environment using Docker.
 2. **Focused Telemetry**
Analyze SSH authentication logs (`/var/log/auth.log`) to capture real attack patterns.
 3. **Detection over Visualization**
Prioritize detection logic and attacker behavior instead of relying only on dashboards.
-

Architecture Overview

The lab was designed as a minimal but functional SOC pipeline:

- **Detection Engine (Brain):** Splunk deployed inside a Docker container
- **Telemetry Source:** Linux system generating SSH authentication logs
- **Data Pipeline:** Log ingestion → indexing → search → detection
- **Access Layer:** Splunk Web Interface

This setup simulates how logs are centralized and processed in modern SOC environments while maintaining simplicity and flexibility.

Phase 1: Building the Detection Engine

To ensure portability and ease of deployment, Splunk was containerized using Docker.

Why Docker?

- Rapid deployment and teardown
- Environment isolation
- Reproducibility across systems
- Lightweight alternative to full VM-based SIEM setups

Once deployed, Splunk became the **core detection engine**, responsible for ingesting and analyzing all incoming telemetry.

Splunk running in Docker container:

```
C:\Windows\System32>docker -v
Docker version 20.10.17, build 10a3c79

C:\Windows\System32>docker ps -a
CONTAINER ID   IMAGE          COMMAND                  CREATED    STATUS    PORTS    NAMES
995d11137de3   splunk/splunk:latest "/sbin/entrypoint.sh..." 6 days ago    Exited (137) 5 days ago    splunk-lab

C:\Windows\System32>docker start 995d11137de3
995d11137de3

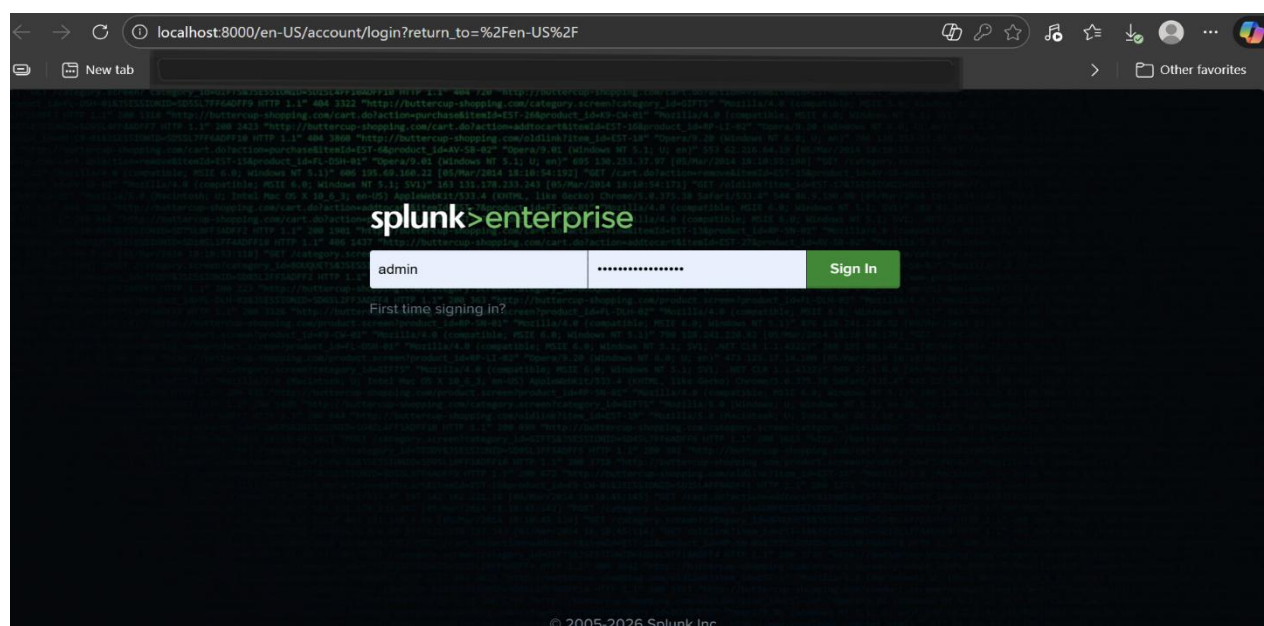
C:\Windows\System32>docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED    STATUS    PORTS    NAMES
995d11137de3   splunk/splunk:latest "/sbin/entrypoint.sh..." 6 days ago    Up 7 seconds (health: starting)    0.0.0.0:8000->8000/tcp, [::]:8000->8000/tcp, 0.0.0.0:9997->9997/tcp, [::]:9997->9997/tcp    splunk-lab

C:\Windows\System32>docker start splunk-lab
splunk-lab

C:\Windows\System32>docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED    STATUS    PORTS    NAMES
995d11137de3   splunk/splunk:latest "/sbin/entrypoint.sh..." 6 days ago    Up 44 seconds (health: starting)    0.0.0.0:8000->8000/tcp, [::]:8000->8000/tcp, 0.0.0.0:9997->9997/tcp, [::]:9997->9997/tcp    splunk-lab

C:\Windows\System32>docker stop splunk-lab
splunk-lab

C:\Windows\System32>docker ps -a
CONTAINER ID   IMAGE          COMMAND                  CREATED    STATUS    PORTS    NAMES
995d11137de3   splunk/splunk:latest "/sbin/entrypoint.sh..." 6 days ago    Exited (137) 5 seconds ago    splunk-lab
```



Phase 2: Establishing the Telemetry Pipeline

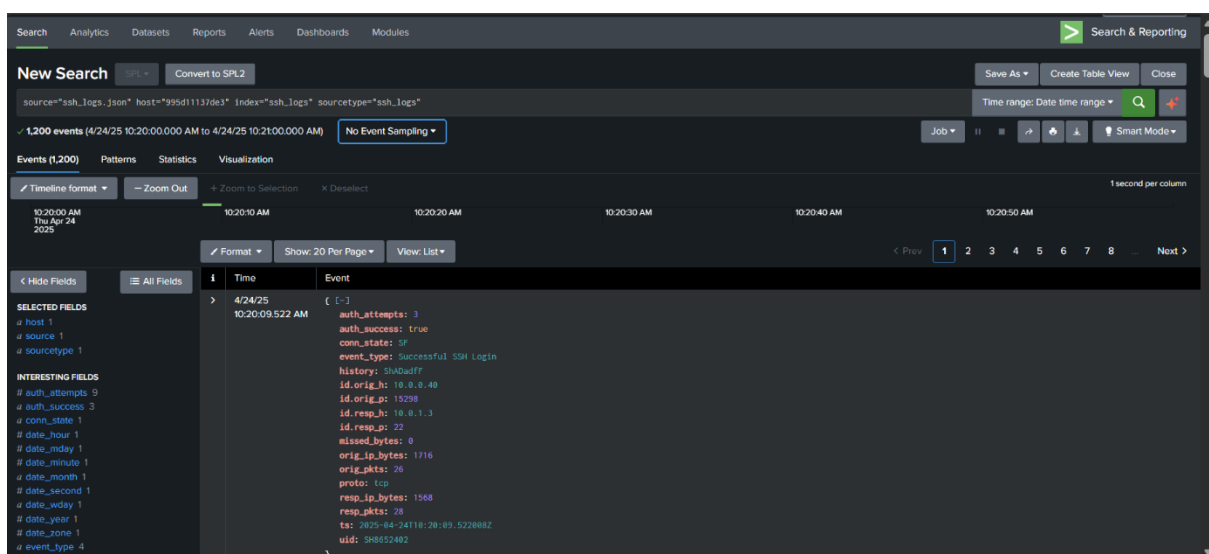
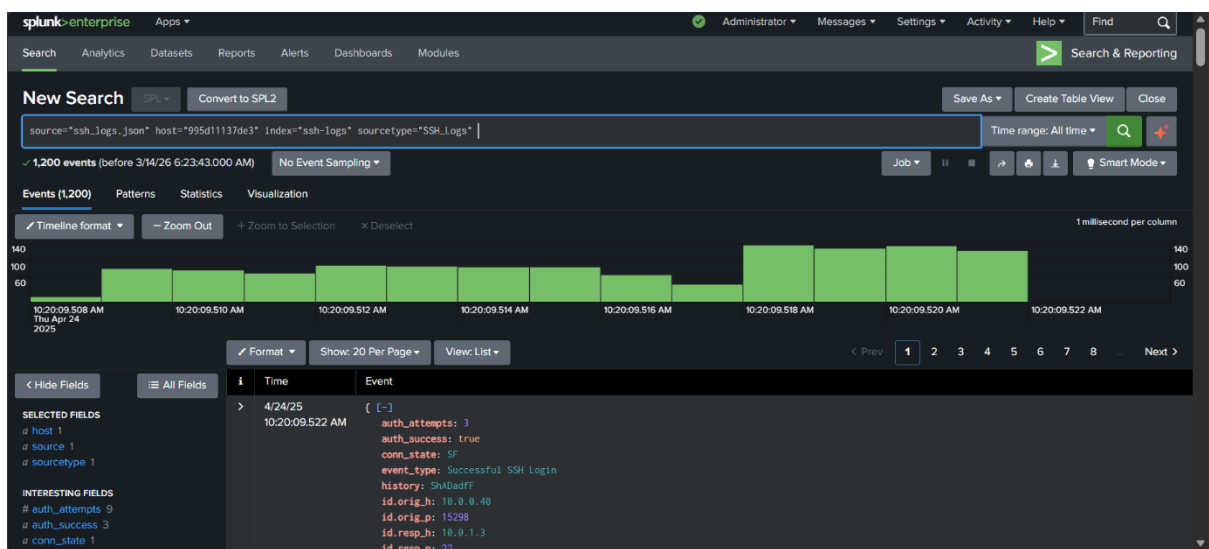
With the detection engine active, the next step was feeding it with meaningful data.

- SSH logs were generated from a Linux system
- Logs were ingested into Splunk under the main index
- Initial validation ensured continuous log flow

At first glance, the logs appeared noisy and repetitive.

But in security, noise is expected — the challenge is identifying **patterns within that noise**.

Raw SSH logs in Splunk:



Phase 3: Detection Logic & Querying

With telemetry flowing, the focus shifted from ingestion to detection.

Detecting Failed Login Attempts

`index=main "Failed password"`

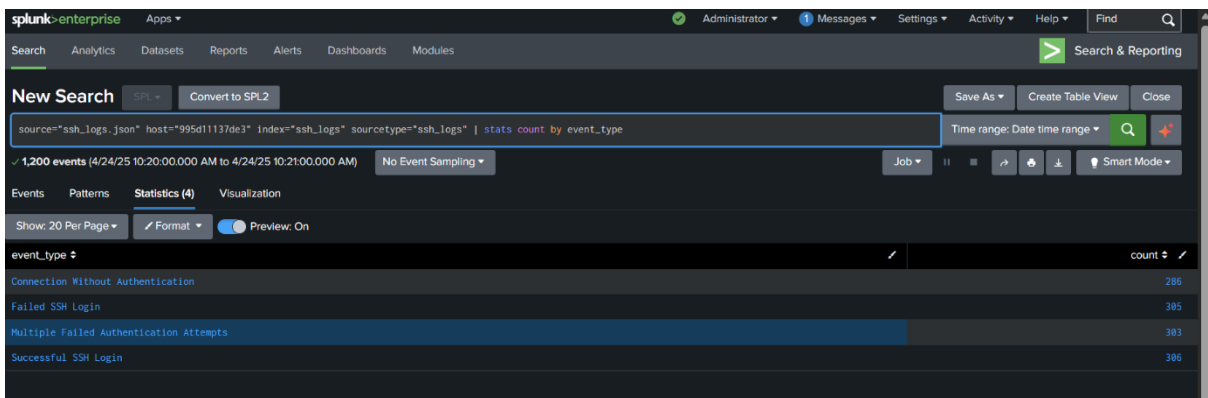
This query revealed repeated authentication failures.

 Insight:

A single failed login is normal.

Multiple failures in rapid succession indicate potential brute-force activity.

 *(Insert Screenshot: Failed login events)*



The screenshot shows the Splunk Enterprise interface with a search query: `source="ssh_logs.json" host="995d11137de3" index="ssh_logs" sourcetype="ssh_logs" | stats count by event_type`. The results show 1,200 events from 4/24/25 10:20:00.000 AM to 4/24/25 10:21:00.000 AM. The statistics table displays the following data:

event_type	count
Connection Without Authentication	286
Failed SSH Login	305
Multiple Failed Authentication Attempts	303
Successful SSH Login	306

🚨 Detecting Multiple Failed Authentication Attempts

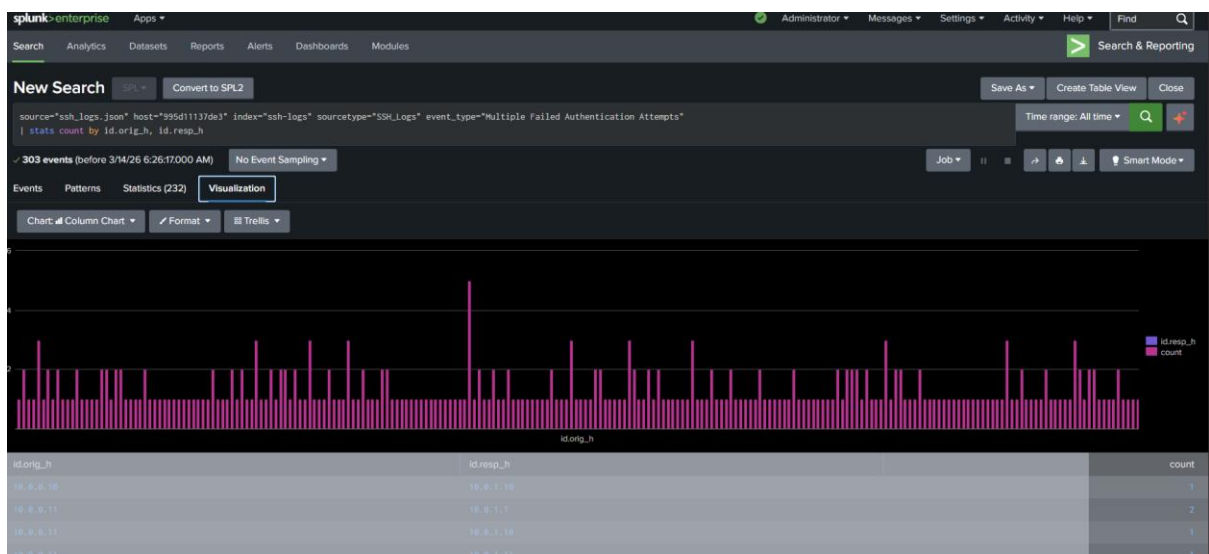
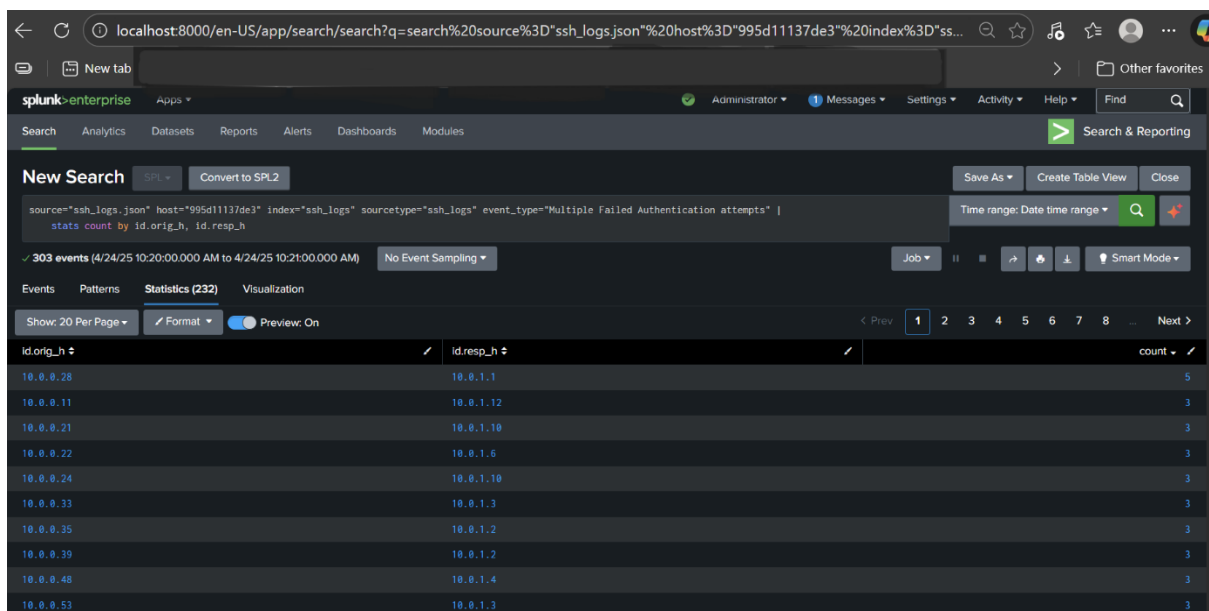
```
index=main "Failed password"
| stats count by src_ip
| where count > 5
| eval event_type="Multiple Failed Authentication Attempts"
```

👉 Insight:

A single failed login attempt is normal in real environments. However, multiple failed attempts from the same IP within a short time window indicate automated brute-force behavior rather than human activity.

By introducing a threshold-based condition, this transforms raw log data into a **behavior-driven detection signal**.

🖥️ (Insert Screenshot: Multiple Failed Authentication Attempts results)



✓ Detecting Successful Logins

```
index=main "Accepted password"
```

This query identified successful authentication events.

👉 Insight:

A successful login following repeated failures may indicate account compromise.

★ Identifying Brute-Force Patterns

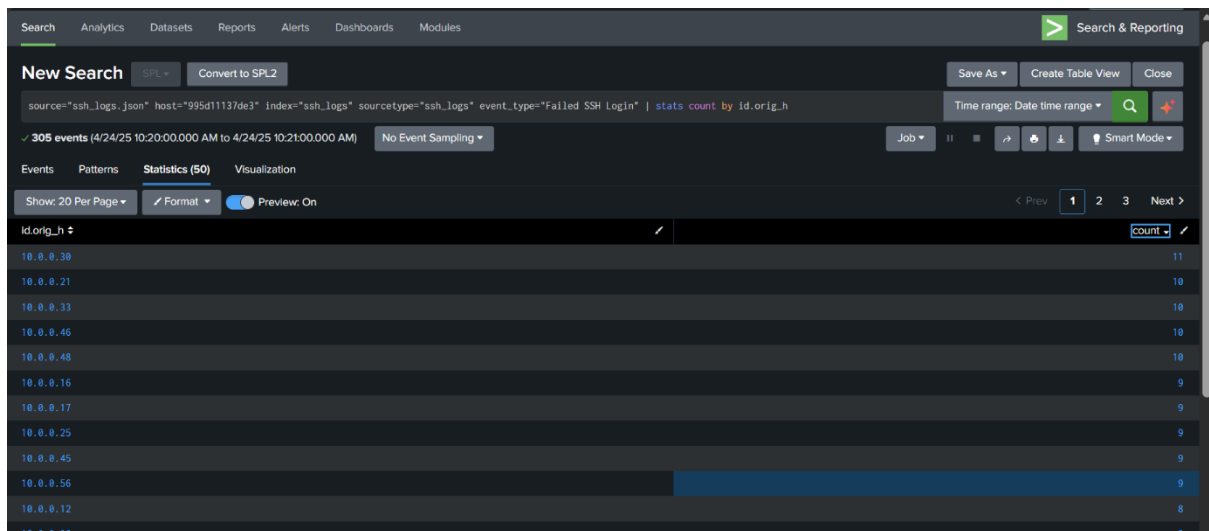
```
index=main "Failed password"
| stats count by src_ip
| sort - count
```

This highlighted IP addresses generating the highest number of failed attempts.

👉 Insight:

A high volume of failures from a single source strongly suggests automated attack tools.

🖼️ *(Insert Screenshot: Top attacking IPs)*



🕒 Time-Based Attack Analysis

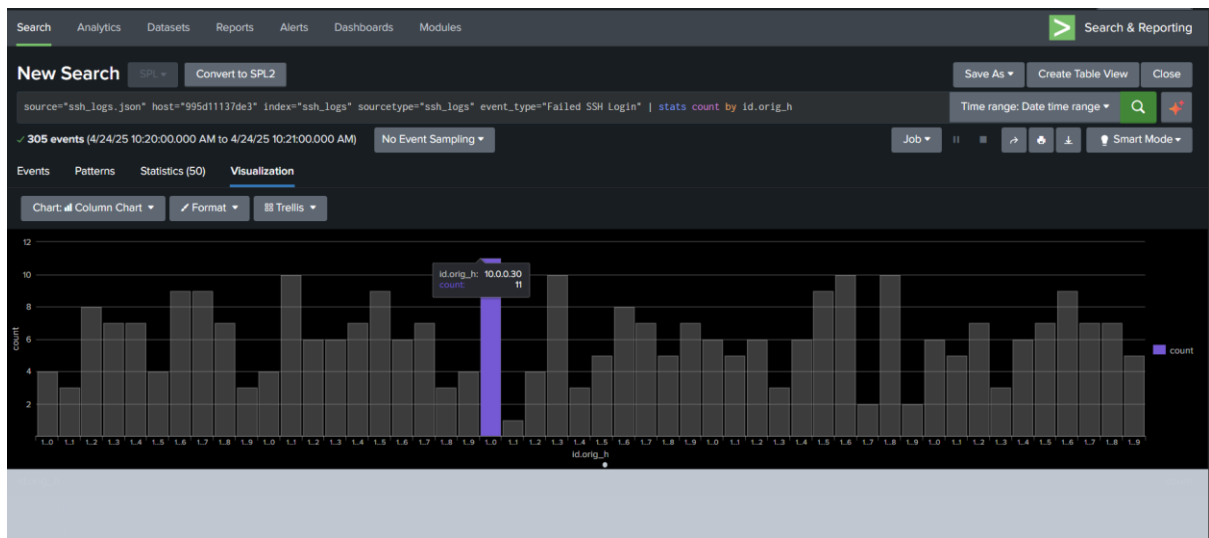
```
index=main "Failed password"
| timechart count
```

This visualized authentication failures over time.

👉 Insight:

Sudden spikes in activity indicate burst-based brute-force attempts.

📊 *Timechart spikes :*



Phase 4: Attacker Behavior Analysis

By correlating the observed data:

- Repeated login failures from a single IP
- High-frequency attempts within short intervals
- Occasional successful authentication after multiple failures

This behavior aligns with:

- **MITRE ATT&CK Technique: T1110 – Brute Force**

At this stage, raw logs are no longer just events — they become **structured attack signals**.



Engineering Challenges & Observations

This project was not purely plug-and-play. Several practical challenges emerged:



Log Noise vs Signal

Raw SSH logs contain a mix of legitimate and malicious activity.

👉 Solution: Focused filtering using targeted queries.

Detection Logic Refinement

Initial queries were too broad and lacked context.

👉 Solution: Introduced aggregation (`stats`) and time-based analysis.

Containerized Deployment Considerations

Running Splunk inside Docker required proper configuration for access and stability.

👉 Solution: Ensured correct port exposure and resource allocation.

Key Findings

- Multiple failed login attempts from specific IP addresses
 - Clear brute-force attack patterns based on frequency and repetition
 - Successful logins observed after repeated failures (potential compromise risk)
 - Attack activity concentrated in short time intervals
-

Security Insights

- SSH services are a common attack vector
 - Weak authentication increases exposure to brute-force attacks
 - Continuous monitoring is essential for early detection
 - SIEM tools enable correlation and behavioral analysis
-

Recommendations

- Implement **SSH key-based authentication**
- Enable **fail2ban** to block repeated failed attempts
- Restrict SSH access using firewall rules
- Configure **Splunk alerts** for:
 - High failed login thresholds
 - Repeated attempts from a single IP
- Enable **multi-factor authentication (MFA)**



Conclusion

This project demonstrates how a **containerized SIEM environment using Splunk** can transform raw SSH logs into actionable security insights.

It highlights the transition from simple log visibility to **detection engineering**, where patterns, timing, and behavior define security events.

This wasn't just about running Splunk — it was about understanding how detection actually works.



What's Next

With the detection pipeline established, the next phase is to move beyond passive analysis:

- Simulating real attacks (e.g., SSH brute-force using Hydra)
- Building automated alerting mechanisms in Splunk
- Expanding log sources for multi-layer detection

The goal is to evolve from observing attacks → to actively engineering detection systems.
